

# MATLAB FUNCTIONS & PROGRAMS

## Chapter 6

Copyright © 2017 STEM Course Prep



# FUNCTIONS

- So far we have covered:
  - functions that calculate and return one value
- Now let's look at:
  - functions that calculate and return more than one value
  - functions that perform a task, such as printing or plotting, without returning any values
- They are different in:
  - the way they are called in the script
  - what the function header looks like

# FUNCTIONS WITH >1 OUTPUT

- Form is similar to the one already covered in class
  - Difference: multiple output arguments will be in the header
- Multiple output arguments must be separated by commas

functionname.m



```
function [arg1,arg2,arg3,...] = functionname(in1,in2,...)
% Comment describing the function
All output variables must be assigned here
end
```

# CALLING THE FUNCTION

- Function returns multiple values
- Function call should be made with a vector with multiple variables on the left hand side
  - Vector in call statement should have the same number of variables as contained in the function output argument list

# FUNCTION CALL EXAMPLE

- Function header:

```
function [x,y,z] = name(a,b,c)
```

- Call statement:

```
[q,h,t] = name(13, 6.9, 4.5);
```

Both must have 3 variables

- Call statement below will only return value for x  

```
result = fnname(11, 4.3);
```

- If in doubt, copy the function header into your script and remove the word function

# EXAMPLE

Script.m

```
rad=input('Enter the radius of the circle:\n');  
[area, circ]=cal_ap(rad)
```

cal\_ap.m

```
function [ a,c ] = cal_ap( r )  
%this function calculates area and circumference  
a=pi*r^2;  
c=2*pi*r;  
end
```

Enter the radius of the circle:

10

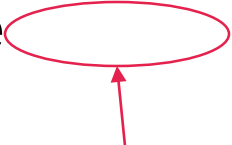
area =314.159265358979

circ = 62.8318530717959

# FUNCTION WITH NO INPUTS

- Occasionally you may have need for a function with no inputs
  - Function may generate random data, etc.
- An example function header:

`function [outarg1, outarg2] = name`



No need for input arguments

- Corresponding function call in script:  
`[oa1, oa2]=name`

# FUNCTIONS WITHOUT OUTPUT

- Functions that do a task without returning values to the script can be created
- Common Uses:
  - Create data files
  - Generate plots
- Function header does not contain:
  - Output arguments
  - =
  - Example: `function name(in1,in2,in3)`
- Call statement in script:
  - Example: `name(in1,in2,in3)`

Notice there are no output arguments or =

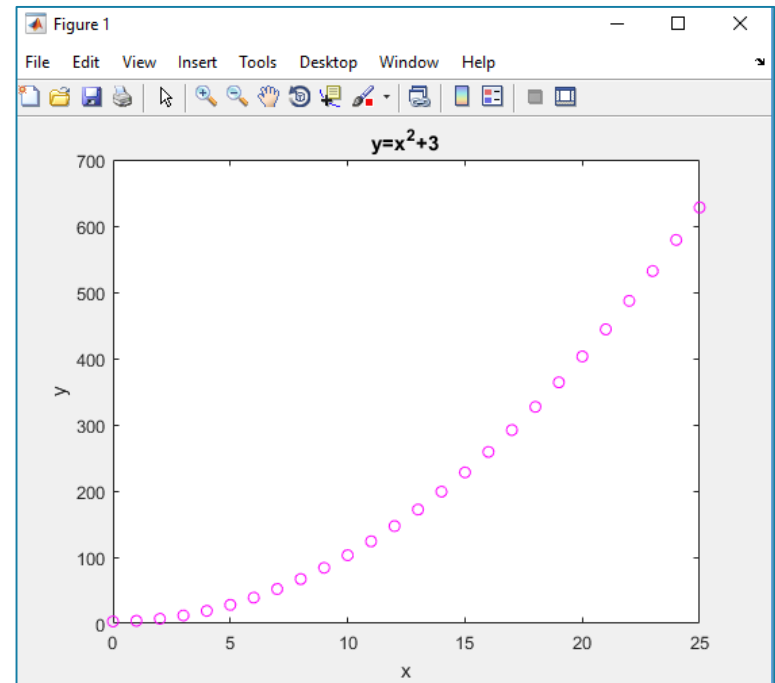
# FUNCTIONS WITHOUT OUTPUT

Script.m

```
x=0:25;  
y=x.^2+3;  
plotxy(x,y)
```

plotxy.m

```
function plotxy( x,y )  
plot(x,y,'mo')  
xlabel('x')  
ylabel('y')  
title('y=x^2+3')  
end
```



# FUNCTION REMINDERS

- Function header and function call must match:
  - name has to be the same (function file name must match too)
  - number of input arguments must be the same
    - functions do not have to have inputs
      - can use () or leave them out
  - number of output arguments must be equal
  - for no output arguments, get rid of = and variables on left hand side
- Functions with output arguments don't display/print values
  - printing values is done in the script

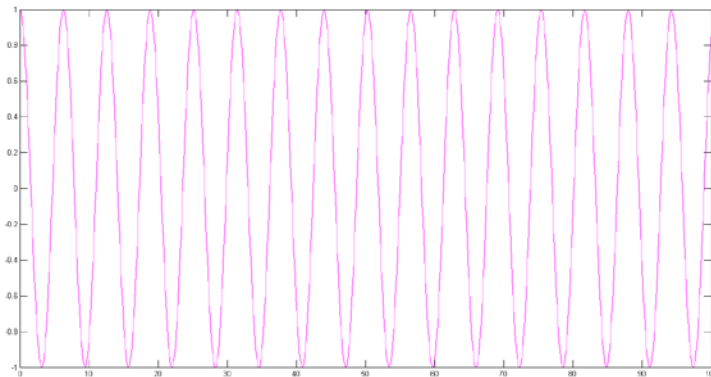
# SUBFUNCTIONS

- Let's say you have a function, f1, that needs to call another function, xyz
  - f1 is the only function that will ever call xyz
  - instead of creating a .m file for xyz, create a subfunction inside f1
- Subfunctions are created inside the main calling function
- Syntax of a subfunction is the same as a stand-alone function
- Why use a subfunction?
  - Eliminates need for another .m file
  - Keeps code in one file
- Subfunctions can only be called by the primary function and other functions contained within the primary function

# SUBFUNCTIONS

plotfunc.m

```
>>plotfunc(0,100)
```



Call to plotfunc results in a plot only

- No need to use  
variable=plotfunc(to,tf)
- time\_generator and trig\_func can only be called from plotfunc

```
function plotfunc( to,tf )
t=time_generator(to,tf);
x=trig_func(t);
plot(t,x, 'mo- ')
end

function [time]=time_generator(t_start,t_stop)
dt=(t_stop-t_start)/1000;
time=t_start:dt:t_stop;
end

function [plotdata]=trig_func(t)
plotdata=cos(t);
end
```

subfunctions

# MODULAR PROGRAM

- Functions written for different tasks
- Main script (program) then calls individual functions to get final results
- Simplest programs consist of function calls to
  - get inputs
  - calculate results
  - display the results
- Benefit of this:
  - Cleaner code
  - Easier to follow
  - Functions can potentially be used for other applications

# EXAMPLE: MODULAR OUTLINE

- In this example modular program, a script calls 3 functions
  - `loaddata` – loads a, b, c from file and returns a, b, c to script  
function `[a,b,c] = loaddata`
  - `calculations` – uses a, b, c to calculate result  
function `result = calculations(a,b,c)`
  - `plotresults` – plots results  
function `plotresults(a,b,c, result)`
- If a, b, c are 3 arrays, script would look like:  
`[a,b,c] = loaddata;`  
`results = calculations(a,b,c);`  
`plotresults(a,b,c,results)`

# REVIEW: LOCAL VARIABLES

- Functions operate in their own workspaces
- Variables used/created in functions are called **local variables**
- Local variables are only available while function is executing
  - Scope is where variables are available
- Command window and scripts operate in the base workspace
  - Variables created in command window and script files are saved in the base workspace
  - These variables are not available to functions unless they are passed in as input arguments

# PERSISTENT VARIABLES

- Let's say you have a function and there's a value you'd like to use next time you call the function
- Two ways to do this:
  - Output argument
  - Create a **persistent variable**
- Memory remains allocated for persistent variables
  - Allows variables to be available next time function is called
- Logical `isempty` to verify variable has been initialized
- **clear functions** resets all persistent variables
- Common use: counter

# EXAMPLE<sup>17</sup>

```
fprintf('Function test has a persistent variable\n')
test % call test function first time
test % call test function second time
```

```
function test
persistent count
if isempty(count)
    count = 0;
end
count = count + 1;
fprintf('count = %d\n',count)
end
```

`isempty(variable)` is a built-in function that tells you if the variable/string is empty

```
>> ch_6_slides
```

Function test has a persistent variable:  
The value of count is 1  
The value of count is 2

count has increased to 2 because the value of count was stored after the first time func2 was called